UNIVERSIDADE ESTADUAL DE CAMPINAS
FACULDADE DE ENGENHARIA MECÂNICA

Relatório Final TG II

Trabalho de Conclusão de Curso

Estudo da segurança de sistemas de autenticação remota por rádio frequência

Autor: **Guilherme de Oliveira Souza**

Orientador: **Prof. Dr. Denis Loubach**

Campinas, novembro de 2016

UNIVERSIDADE ESTADUAL DE CAMPINAS
FACULDADE DE ENGENHARIA MECÂNICA

Relatório Final TG II

Trabalho de Conclusão de Curso

Estudo da segurança de sistemas de autenticação remota por rádio frequência

Autor: **Guilherme de Oliveira Souza**

Orientador: **Prof. Dr. Denis Loubach**

Curso: Engenharia de Controle e Automação

Trabalho de Conclusão de Curso apresentado à Comissão de Graduação da Faculdade de Engenharia Mecânica, como requisito para a obtenção do título de Engenheiro de Automação e Controle.

Campinas, 2016

S.P. – Brasil

Agradecimentos

Este trabalho não poderia ser terminado sem a ajuda do meu orientador Professor Doutor Denis Loubach, com seus comentários diretos e sinceros.

A Dan Boneh, professor da Universidade de Stanford e instrutor do excelente (e muito recomendado) curso online de criptografia I pela plataforma Coursera, que me guiou pelo caminho das pedras do mundo da cripto.

Aos especialistas e entusiastas, como Solar Designer, Thomas Pornin, Matthew Green e Adam Langley, nas listas de e-mails da PHC (password-hashing competition), no fórum Cryptography Stack Exchange e em vários blogs online, por toda a paixão de aprender e compartilhar que possuem.

Aos meus pais, Eduardo e Irany, por dar tanto de suas vidas para o meu futuro, mesmo tão longe.

Índice

Agradecimentos i

Índice ii

Resumo 1

Lista de Figuras 2

Lista de Tabelas 3

Nomenclatura 3

Subscritos 3

Abreviações 3

Siglas 3

Operadores 4

Capítulo 1 – Introdução 5

Objetivos 5

Capítulo 2 - Definição 7

Topologia e Restrições de Construção 7

Propriedades Desejadas 9

Modelo de Ameaça 9

Capítulo 3 – Exemplos Comerciais 11

Código Fixo 11

Ataque de Busca Exaustiva 11

Ataque de Busca Exaustiva Melhorado 12

Classic KeeLoq® 13
Descrição do Protocolo 14
Fase de Utilização 14
Fase de Aprendizagem 17
Ataques 17
Ataques teóricos na cifra 18
Ataque prático na implementação do hardware 19
Ataque prático na implementação em software 20
Ataque prático de repetição 21
Ultimate KeeLoq® 22
Descrição do Protocolo 23
Fase de Utilização 23
Fase de Aprendizagem 26
Ataques 27
Capítulo 4 – Resultados e Discussões 28
Análise Código Fixo 28
Análise Classic KeeLoq® 28
Análise Ultimate KeeLoq® 29
Capítulo 5 – Sugestão de Novo Protocolo 31
Protocolo SCRAM 31
Adequando SCRAM 33

Capítulo 6 – Programação VHDL 36

Fundamentos 36

Código Fixo 38

Classic KeeLoq® 38

SCRAM Modificado 41

Capítulo 7 – Conclusões 45

Bibliografia 48

Resumo

SOUZA, Guilherme de Oliveira, *Estudo da segurança de sistemas de autenticação remota por rádio frequência*, Faculdade de Engenharia Mecânica, Universidade Estadual de Campinas, Trabalho de Conclusão de Curso, (2016).

Sistemas de autenticação remota por rádio frequência tem diversas aplicações nos dias de hoje, como por exemplo: portões eletrônicos, travas de automóveis e cartões de acesso RFID. De modo geral, eles são usados para proteger valores (sejam materiais, intelectuais ou de qualquer outra forma) contra acessos indevidos. Com o crescimento na adoção e nos valores protegidos desses dispositivos, cresce a necessidade de garantir um nível de segurança adequado desses sistemas contra acessos não autorizados. Este trabalho define esses sistemas, com a descrição de um modelo de ameaça, limitações de construção e as propriedades desejadas para que sejam considerados seguros. Algumas soluções comerciais do passado e presente são exploradas, descrevendo alguns ataques conhecidos e possíveis falhas.

Palavras Chave: autenticação remota, rádio frequência, segurança, fixed code, rolling code, hopping code, remote keyless system, KeeLoq

Lista de Figuras

Figura 1 - Diagrama da topologia 7

Figura 2 - Central Eletrônica CP2010. Fonte: http://www.peccinin.com.br/produto/cp-2010/ 8

Figura 3 - Transmissor TX3C. Fonte: http://www.peccinin.com.br/produto/tx-3c/ 8

Figura 4 - Uma chave de carro e, sob escala, bateria A23 (em cima) e CR2032 (embaixo) 9

Figura 5 - Exemplo de transmissor de código fixo 11

Figura 6 - Ilustração de um “bit shift register” de tamanho 5 12

Figura 7 - Diagrama do pacote de utilização 15

Figura 8 - Diagrama do pacote de utilização 24

Figura 9 - Forma de cálculo do código de autenticação 25

Figura 10 - Diagrama do pacote de aprendizado 27

Figura 11 - Sequência de mensagens do protocolo SCRAM 32

Figura 12 - Sequência de mensagens do protocolo baseado em SCRAM 34

Figura 13 - Pacotes início do cliente, início do servidor e fim do cliente 35

Figura 14 - Diagrama das entidades implementadas 36

Figura 15 - Exemplo da transmissão dos bits 01000111 37

Figura 16 - Máquina de estados do receptor de codificação PWM 38

Figura 17 - Hierarquia dos componentes na implementação KeeLoq.....39

Figura 18 - Máquina de estados da implementação do periférico KeeLoq 40

Figura 19 - Máquina de estados da implementação da central KeeLoq 40

Figura 20 - Hierarquia dos componentes na implementação SCRAM modificado.....41

Figura 21 - Funcionamento do SHA1 PRNG.....42

Figura 22 - Máquina de estados da implementação do cliente SCRAM 43

Figura 23 - Máquina de estados da implementação do servidor SCRAM.....44

Lista de Tabelas

Tabela 1 - Lista de alguns ataques conhecidos à cifra KeeLoq 18

Tabela 2 - Tempo de busca exaustiva para cada tamanho de seed.....21

Tabela 3 - Comparação dos protocolos 45

Nomenclatura

Subscritos

TX Como armazenado pelo transmissor

RX Como armazenado pelo receptor

Abreviações

TX Transmissor

RX Receptor

Siglas

TLS	Transport Layer Security
SCRAM	Salted Challenge Response Authentication Mechanism
MAC	Message Authentication Code
AE	Authenticated Encryption

Operadores

$a \mid b$	Concatenação
$a \oplus b$	XOR

Capítulo 1 – Introdução

Sistemas de autenticação por rádio frequência são amplamente utilizados em nossa sociedade. As duas aplicações mais comuns são: portão de garagem e chave de carro.

Atualmente, parece haver grande incentivo para desenvolver formas de burlar esses sistemas, devido ao aumento dos valores que protegem. No entanto, boa parte das instalações desses sistemas ainda parece adotar soluções simplórias, com ataques práticos já conhecidos, divulgados em outros estudos acadêmicos e até mesmo em blogs de interessados no assunto.

Por outro lado, já existem sistemas de autenticação desenvolvidos e adotados com sucesso em outras aplicações, como por exemplo TLS, SCRAM e OAuth. O que motiva esse estudo é entender as limitações práticas em aplicar esses protocolos já conhecidos em sistemas baseados em radiofrequência e, se possível, propor alterações que o tornem mais adequados às necessidades nesse domínio.

Esse estudo é relativo à primeira parte do trabalho de graduação (TG I). A segunda parte (TG II) será desenvolvida no segundo semestre de 2016.

Objetivos

O principal resultado dessa primeira parte do estudo é conhecer em profundidade as soluções comerciais de hoje, mostrando seus pontos fracos e fortes. Serão utilizados como base a documentação dos fabricantes, os estudos acadêmicos e publicações de entusiastas amadores.

O principal resultado do estudo final será uma proposta prática para o sistema de autenticação baseado em tecnologias reconhecidas como seguras em outras aplicações. Essa proposta inclui a descrição do protocolo e dos componentes de hardware necessários para sua implementação. Também serão apresentadas estimativas de custos do hardware

e dados de desempenho, como duração da bateria, alcance máximo, peso do transmissor, etc.

Capítulo 2 - Definição

Esse capítulo irá definir precisamente o objeto de estudo desse texto, bem como descrever o escopo de segurança adotado.

Topologia e Restrições de Construção

Os sistemas de autenticação remota por rádio frequência aqui considerados seguem o modelo de cliente-servidor. O receptor assume o papel de servidor; os transmissores são os clientes. Cada instalação é composta de um único receptor e um conjunto de múltiplos transmissores, como mostrado na Figura 1.

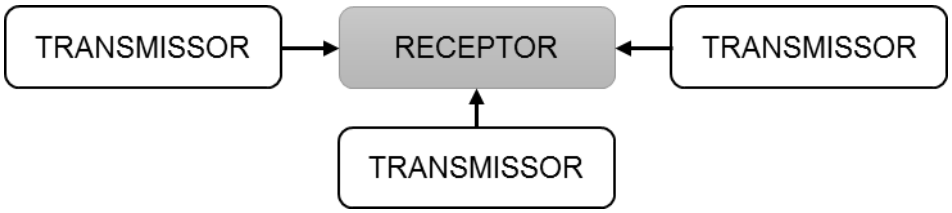

Figura 1 - Diagrama da topologia

O receptor é responsável por autenticar o acesso de um transmissor, executando a ação requerida por ele, por exemplo: desativar um alarme. O receptor se comporta como servidor pois controla o recurso compartilhado, por exemplo: alarme. Os transmissores se comportam como clientes pois são eles que iniciam a comunicação.

Os transmissores são pequenos dispositivos, geralmente carregados por uma pessoa, capaz de enviar comandos ao receptor. Cabe ao transmissor convencer o receptor de que sua requisição é autêntica, resultando na execução da ação desejada. A Figura 2 mostra um exemplo comercial de um receptor alojado numa central eletrônica e a Figura 3 o de um transmissor.

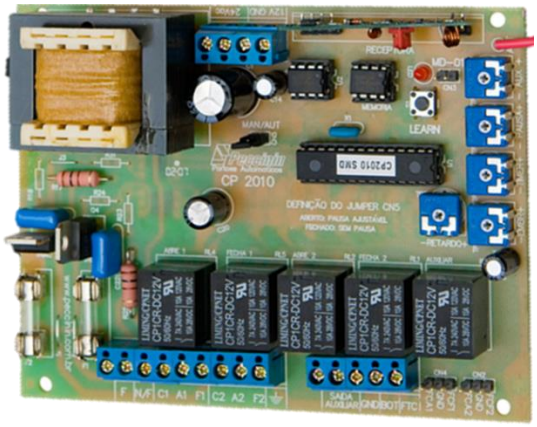

Figura 2 - Central Eletrônica CP2010.
Fonte: http://www.peccinin.com.br/produto/cp-2010/

Figura 3 - Transmissor TX3C.
Fonte: http://www.peccinin.com.br/produto/tx-3c/

Em algumas configurações, o receptor guarda uma lista de transmissores conhecidos. A etapa de cadastro de um novo transmissor frente a um receptor envolve o compartilhamento de algum segredo, utilizado para autenticar as futuras requisições.

A comunicação entre os elementos é geralmente de mão-única. Embora seja possível imaginar sistemas que façam uso de uma comunicação duas vias (duplex), é incomum encontra-los, provavelmente por motivos de simplicidade.

As restrições físicas são mais severas para os transmissores, por serem pequenos dispositivos que precisam ser carregados e manipulados com facilidade pelo usuário. Eles geralmente apresentam dimensões de cerca de 5 x 4 x 2 cm (chave de carro típica) e utilizam pilhas com carga entre 55mAh (modelo A23) e 225mAh (modelo CR2032), representadas na Figura 4.

Figura 4 - Uma chave de carro e, sob escala, bateria A23 (em cima) e CR2032 (embaixo)

O receptor tem uma fonte mais abundante de energia, seja uma bateria de carro ou a própria rede elétrica. As restrições nas dimensões também são bem mais relaxadas.

Propriedades Desejadas

Um sistema de autenticação funcional deve atender aos seguintes requisitos:

- Deve haver um modo de cadastro de um novo transmissor;
- Somente transmissores previamente cadastrados devem conseguir se autenticar com sucesso;
- Os transmissores com múltiplas ações, geralmente associadas a botões diferentes, devem podem transmitir a informação de qual ação deve ser desempenhada. Essa transmissão deve ser sigilosa (nenhuma outra parte deve tomar ciência de qual ação foi escolhida) e íntegra (nenhuma outra parte deve conseguir alterar o comando enviado);
- Um receptor pode ter vários transmissores cadastrados ao mesmo tempo;

Modelo de Ameaça

O modelo de ameaça define quais poderes são concedidos a um adversário, de modo que se deixe claro o escopo de segurança considerado.

Dessa forma, os poderes concedidos a qualquer agente com o intuito de subverter o sistema de autenticação são:

- Ter conhecimento de toda a arquitetura e funcionamento do hardware e protocolo. Isso implica que a segurança do sistema não pode se basear em obscuridade;
- Interceptar e escutar qualquer comunicação entre os componentes. Isso implica que o conteúdo da comunicação não pode revelar nenhum segredo;
- Alterar qualquer comunicação entre as partes, seja adicionando, removendo ou alterando o conteúdo. Isso implica que deve haver algum mecanismo para verificar a integridade da transmissão;
- Agir como elemento do sistema, por exemplo: se comunicar com o receptor como se fosse um transmissor válido.
- Ter acesso a dispositivos do mesmo fabricante em instalações diferentes. Isso implica que cada instalação deve ser independente e não deve haver nenhum segredo compartilhado em elas.

Como resultado desse modelo de ameaça, esse estudo não considera ataques envolvendo:

- Acesso físico a elementos do sistema. Por exemplo: não é permitido a um adversário simplesmente tomar posse de um transmissor cadastrado e usá-lo;
- Engenharia social, de forma a enganar usuários do sistema. Por exemplo: convencer um usuário legítimo a usar o sistema de forma a beneficiar o adversário. Outro exemplo desse tipo de ataque (conhecido como *jammer*) é enviar um ruído na frequência de operação em direção ao receptor quando um transmissor envia a ordem de ativar um alarme, de forma a bloquear a comunicação. Um usuário distraído que não perceba a falha do comando irá deixar o alarme desativado.

Capítulo 3 – Exemplos Comerciais

Este capítulo apresenta exemplos de três soluções comerciais encontradas. Cada solução será descrita em detalhes e depois serão apresentados ataques teóricos e práticos, relatados em diversos trabalhos acadêmicos e amadores.

Código Fixo

Um design simples de um protocolo de autenticação seria usar um código fixo, conhecido pelo transmissor e receptor. O transmissor envia esse mesmo código a cada utilização e o receptor confere se o código recebido corresponde com algum dos cadastrados em sua memória.

O pacote de transmissão pode conter algumas informações adicionais, como um identificador do botão pressionado, indicador de bateria baixa, etc. Esse conteúdo extra não é criptografado nem protegido contra alterações intencionais.

Nesse esquema, o receptor tem uma lista com os códigos dos transmissores cadastrados. O código geralmente contém de 8 a 12 bits e é configurado no transmissor, usualmente com o uso de um “DIP switch” [1, p. 26]. A Figura 5 retrata um transmissor de código fixo de 12 bits, configurável através das chaves de seleção no canto inferior esquerdo.

Figura 5 - Exemplo de transmissor de código fixo

<u>*Ataque de Busca Exaustiva*</u>

Como o comprimento do código (8 a 12 bits) é bem curto, um ataque de força bruta, enviando todas as combinações de código possíveis para o receptor, pode ser executado com sucesso em um tempo bem pequeno.

Por exemplo, para enviar todos as possíveis combinações de 12 bits são necessários 2^{12} = 4096 transmissões de 12 bits. Considerando uma taxa de 2ms/bit e 2ms de espera entre as transmissões, chega-se a um tempo total de 1 minuto e 47 segundos.

Ataque de Busca Exaustiva Melhorado

Alguns receptores utilizam um “bit shift register” para armazenar o código recebido e compará-lo com os códigos conhecidos [1, p. 30].

A Figura 6 ilustra o funcionamento de um “bit shift register” com 5 bits de tamanho. A cada ciclo de “clock”, os bits são movidos uma casa adiante, jogando fora o bit mais antigo e admitindo um novo bit na outra extremidade.

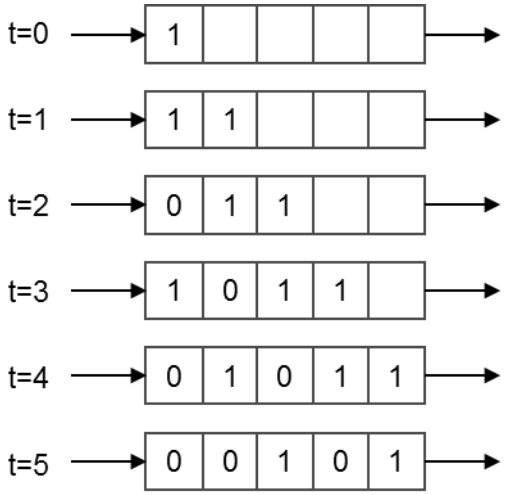

Figura 6 - Ilustração de um “bit shift register” de tamanho 5

Essa construção é comumente empregada por permitir simplificações no protocolo e hardware. O protocolo não precisa utilizar cabeçalhos para definir o começo de cada pacote e o transmissor pode emitir o código repetidas vezes, de forma a diminuir a chance de ruídos atrapalharem a comunicação.

O receptor então verifica o conteúdo do registrado a cada ciclo, considerando somente os últimos N bits recebidos. O ataque descrito na seção anterior pode se aproveitar dessa característica. Para isso, a sequência de bits utilizada no ataque deve ser cuidadosamente escolhida de forma que, a cada novo bit, os últimos N bits formem uma sequência ainda não testada.

Matemáticos provaram a existência e descreveram formas para obter essa sequência, conhecida como sequência De Bruijn [2]. Como exemplo, para $N = 3$, pode-se obter a sequência 0001011100. Cada conjunto de 3 bits consecutivos forma um padrão único: 000, 001, 010, 101, 011, 111, 110, 100. O número de bits numa sequência De Bruijn de forma que cada N bits consecutivos forme um padrão único é dado por $2^N + N - 1$.

Refazendo o cálculo do tempo de ataque para testar todas as possíveis combinações de 12 bits, são necessários $2^{12} + 12 - 1 = 4107$ bits. Considerando a mesma taxa de transmissão (2ms/bit), o tempo total cai de 1 minuto e 47 segundos para 8,2 segundos.

Um exemplo de implementação prática desse ataque é o OpenSessame, utilizando um brinquedo da Mattel® como plataforma de hardware [3]. O código fonte e tutoriais foram disponibilizados abertamente e o hardware pode ser adquirido por cerca de $12.

Classic KeeLoq®

KeeLoq é uma tecnologia proprietária adquirida pela empresa Microchip em 1995 [4].

Implementada em hardware dedicado e disponível comercialmente até os dias de hoje em algumas famílias de chip como HCS2xx/3xx/4xx/5xx [5].

O protocolo baseado em KeeLoq aplica o conceito de "hopping code", onde o código enviado pelo transmissor é diferente a cada utilização e de uso único. No momento do envio, esse código é gerado pelo transmissor com base num contador sequencial, incrementado a cada uso. O valor desse contador é cifrado pelo transmissor usando o algoritmo de cifra em bloco de mesmo nome, o KeeLoq.

Para que o protocolo funcione, o receptor e o transmissor precisam ter conhecimento de um segredo comum (referido como chave do dispositivo). Esse segredo é compartilhado numa etapa de aprendizado, executado previamente, onde o receptor toma conhecimento do transmissor.

Como o segredo nunca é enviado abertamente entre as partes, o uso da técnica de "hopping code" dificulta um atacante ter acesso a ele. Além disso, como o código é de uso único, não é possível reutilizar uma comunicação anterior.

Embora esse protocolo represente um avanço em relação ao simples envio de um código fixo, ele apresenta várias falhas de segurança teóricas e práticas, resultante do design e implementação.

As subseções seguintes descrevem o protocolo em mais detalhes e apresenta algumas falhas conhecidas.

Descrição do Protocolo

O protocolo é composto de duas fases: aprendizado e utilização.

A fase de aprendizagem tem por objetivo combinar um segredo entre o receptor e o transmissor e é executada apenas uma vez. Em alguns casos (como chave de automóvel), essa etapa é realizada antes da venda ao consumidor final.

A fase de utilização é executada pelo usuário a cada vez que pressiona um botão de ação.

Como mostrado, o protocolo é implementado em diversas famílias de chips, com pequenas variações entre eles. Esse estudo se foca em uma dessas variantes.

Fase de Utilização

A descrição dada nessa subseção toma como base o funcionamento do chip HCS410, com serial de 28bits [6].

O transmissor armazena as seguintes informações:

- $SERIAL_{TX}$: valor de 28 bits, globalmente único
- KEY_{TX}: chave de encriptação de 64 bits
- $COUNTER_{TX}$: contador de uso de 16 bits, inicialmente em zero

A cada envio, um pacote de 69 bits com o formato mostrado na Figura 7 é gerado pelo transmissor.

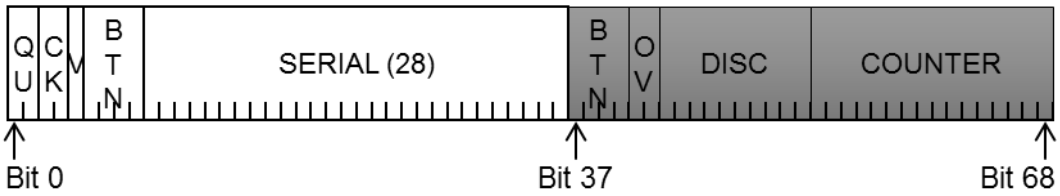

Figura 7 - Diagrama do pacote de utilização

Os 32 bits de 37 a 68 são cifrados com KeeLoq usando a chave KEY_T.

Os campos presentes no pacote são:

- QU: número de vezes que o botão foi pressionado:
  - $QU = 00$ para pressionamento simples
  - $QU = 01$ para pressionamento duplo
  - $QU = 10$ para pressionamento triplo
  - $QU = 11$ para pressionamento mais que triplo
- CK: 2 bits verificadores para detectar alterações acidentais na transmissão. É importante ressaltar que sua função não é frustrar alterações maliciosas.
- V: se $V = 1$, a bateria do transmissor está acabando
- BTN: 4 bits com o estado de cada um dos 4 botões. Se $BTN[i] = 1$, o i-ésimo botão está pressionado
- $SERIAL = SERIAL_{TX}$

- OV: 2 bits de overflow do contador, cujo funcionamento exato não foi elaborado pelo fabricante
- $DISC = SERIAL_{TX}[18:27]$, os 10 bits menos significativos de $SERIAL_T$, utilizado pelo receptor para verificar a integridade do conteúdo cifrado.
- $COUNTER$: valor atual de $COUNTER_{TX}$

O receptor, por sua vez, armazena as seguintes informações:

- TS_{RX}: conjunto de transmissores cadastrados. Cada elemento contém:
    - $SERIAL_{RX}$: identificador de 28 bits do transmissor
    - KEY_{RX}: chave de encriptação de 64 bits
    - $COUNTER_{RX}$: último valor recebido do contador de uso de 16 bits

O receptor somente executa a ação se o pacote recebido passar pelas seguintes validações em ordem:

1. Valor dos bits verificadores em CK confere com o esperado;
2. Existe algum transmissor cadastrado com o $SERIAL$ recebido;
3. Ao decriptar os últimos 32 bits do pacote com a chave KEY_{RX}, o conteúdo de $DISC$ confere com os 10 bits menos significativos de $SERIAL$. Essa verificação permite concluir que há alta chance de o conteúdo cifrado ter sido alterado em trânsito e que o transmissor usou a chave correta $KEY_{TX} = KEY_{RX}$;
4. Contador recebido é bem próximo do esperado: $COUNTER_{RX} < COUNTER \leq COUNTER_{RX} + 16$. Isso permite até 16 ativações acidentais entre duas ativações válidas.

Caso a etapa 4 falhe, mas $COUNTER_{RX} < COUNTER \leq COUNTER_{RX} + 32768$, o valor de $COUNTER$ é salvo temporariamente. Se o próximo pacote recebido trouxer $COUNTER + 1$ como valor do contador, o pacote é tido como válido. Essa resincronização permite o sistema se recuperar de grandes desvios no valor contador.

Se o pacote for aceito, o valor de $COUNTER_{RX}$ é atualizado para $COUNTER$. Efetivamente tornando inválido qualquer outro pacote com um valor de contador menor.

Fase de Aprendizagem

A descrição dada nessa subseção toma como base o funcionamento do chip HCS500, com aprendizagem baseado no serial [7].

Essa fase deve ser executada antes da utilização de um novo transmissor. O receptor é colocado no modo de aprendizado e o transmissor deve emitir um pacote de utilização. O serial do novo transmissor é lido do pacote e utilizado para determinar a chave de dispositivo. Esse procedimento é chamado de derivação de chave.

Existem três esquemas de derivação sugeridos pela Microchip [7, p. 17]. O esquema mais utilizado é função apenas do serial (28bits) e da chave do fabricante (64bits) [8, p. 167]. A chave do fabricante KEY_{FAB} é gravado em cada dispositivo do sistema no momento de sua produção. A chave do transmissor é então calculada por (1):

$$\begin{aligned} KEY_{TX} &= KEY_{RX} \\ &= DEC(KEY_{FAB}; PAD_1 \mid SERIAL) \mid DEC(KEY_{FAB}; PAD_2 \mid SERIAL) \end{aligned} \tag{1}$$

Onde

- $DEC(KEY;\ CIPHERTEXT)$ é o algoritmo de decriptação KeeLoq
- PAD_1 e PAD_2 são valores fixos
- $\mid$ é o operador de concatenação

O receptor então usa essa chave KEY_{RX} para decriptar os últimos 32 bits do pacote. Se o conteúdo de $DISC$ conferir com os 10 bits menos significativos de $SERIAL$, essa etapa ocorreu com sucesso. O receptor então cria um novo registro no conjunto TS_{RX}, com $SERIAL_{RX} = SERIAL$ e $COUNTER_{RX} = COUNTER$.

Ataques

O protocolo KeeLoq apresenta vulnerabilidades conhecidas há vários anos, publicadas por pesquisadores e amadores. Essa sessão irá apresentar algumas delas.

Ataques teóricos na cifra

A cifra de bloco KeeLoq, utilizada no protocolo, foi alvo de várias criptoanálises pela academia, com resultados cada vez mais promissores em enfraquecer a cifra, descrevendo ataques teóricos com menor complexidade de execução, memória e dados.

Há um ataque capaz de obter a chave de encriptação ao analisar 2^{16} pares $(TEXTO; ENC(KEY; TEXT))$, onde ENC é a função de encriptação KeeLoq [9]. A complexidade de tempo do ataque descrito é de $2^{44.5}$ execuções da rotina de encriptação.

Na prática, obter os pares necessários para o ataque é difícil. Porém, dado que os pares sejam dados, o ataque pode ser calculado em 7.8 dias usando uma CPU de 64 núcleos.

O mesmo artigo lista outros ataques teóricos anteriormente publicados, parcialmente reproduzida na Tabela 1:

Tabela 1 - Lista de alguns ataques conhecidos à cifra KeeLoq

Tipo de ataque	Ano	Complexidade			Referência
		Dado[1]	Tempo[2]	Memória	
Time-Memory Trade-Off	1980	2CP	$2^{42.7}$	~100TB	[10]
Slide/Algebraic	2007	2^{16}KP	$2^{51.4}$	-	[11]
Slide/Guess-and-Determine	2007	2^{32}KP	2^{52}	16GB	[12]
Slide/Guess-and-Determine	2007	2^{32}KP	$2^{50.6}$	16GB	[13]
Slide/Cycle Structure	2007	2^{32}KP	$2^{39.4}$	16.5GB	[11]
Slide/Fixed Points	2008	2^{32}KP	2^{27}	>16GB	[14]
Slide/Meet-in-the-Middle	2010	2^{16}KP	$2^{44.5}$	~3MB	[9, p. 10]

Ataque prático na implementação do hardware

Kasper descreve um ataque utilizando análise de potência para obter a chave de encriptação de um transmissor. O ataque mede o perfil de consumo de corrente do circuito integrado da Microchip no pino de aterramento utilizando uma taxa de amostragem de 200

[1] Número de pares (TEXTO; ENC(KEY; TEXTO)) necessários para executar o ataque. Os pares podem ser de “Chosen Plaintext” (CP) ou “Known Plaintext” (KP). No primeiro caso, o atacante tem o poder de escolher o valor de TEXTO, no segundo, não.

[2] Medido no número de execuções completes da cifra KeeLoq

MHz. O autor relata sucesso em obter a chave do dispositivo após capturar seis execuções da fase de utilização [8, pp. 172-174].

O modelo de ameaça considerado neste presente trabalho não permite ao atacante ter acesso físico a dispositivos do sistema. Portanto, esse ataque não seria possível, já que não haveria como fazer as medições necessárias.

Ataque prático na implementação em software

Kasper utiliza análise de potência para obter a chave do fabricante a partir de um receptor. O trabalho relata que a lógica do KeeLoq é implementada em software e utiliza a chave do fabricante durante a fase de aprendizado. O ataque é conduzido com sucesso utilizando uma taxa de amostragem de 125MHz em apenas uma execução da fase de aprendizagem [8, p. 184].

Uma vez que a chave do fabricante é extraída de um receptor, ela pode ser usada para clonar qualquer transmissor do mesmo fabricante, bastando capturar duas comunicações da fase de utilização [8, p. 187]. A captura dessas comunicações pode ser realizada a uma distância de algumas centenas de metros e o hardware necessário é bem simples e barato (um receptor de rádio frequência e um chip com a chave do fabricante e a implementação do algoritmo de derivação).

Essa falha também deixa claro que o fabricante pode tão facilmente clonar qualquer transmissor já em funcionamento.

O protocolo prevê uma mitigação a essa falha, onde a chave do dispositivo é gerada a partir da combinação do serial com um valor aleatório, único por transmissor, denominado “seed”. Ao contrário do serial, que é reenviado em toda comunicação, o “seed” só é enviado na fase de aprendizado, ao pressionar uma sequência específica de botões. Existem três versões implementadas nos chips comerciais, variando o tamanho do “seed”: 32, 48 ou 60 bits.

O uso de “seed” dificulta a clonagem, adicionando o tempo de executar uma busca exaustiva nos possíveis valores para encontrar a chave do dispositivo correta. A Tabela 2 apresenta o tempo dessa busca, no pior caso, utilizando hardware especializado chamado COPACOBANA [8, p. 191]:

Tabela 2 - Tempo de busca exaustiva para cada tamanho de seed

Tamanho do seed (bits)	**Tempo da busca**
32	0.33 segundos
48	5.9 horas
60	1011 dias

O uso de um “seed” de 60 bits consegue adicionar margem de segurança significativa ao sistema. Porém, essa técnica é raramente utilizada em produtos comerciais, não sendo encontrada em nenhum dispositivo analisado.

Ataque prático de repetição

Outro ataque prático, reportado [15] e aperfeiçoado [1] posteriormente, não leva em conta a criptografia que sustenta o protocolo. O dispositivo montado como prova de conceito custou menos de $32 e tem dimensões menores do que um aparelho celular.

O ataque consiste em posicionar o aparelho perto do receptor, de forma a sequestrar um pacote da etapa de utilização. Esse pacote pode ser posteriormente utilizado pelo atacante para burlar o sistema (e destravar o carro, por exemplo).

O ataque funciona da seguinte forma:

1. O atacante posiciona o aparelho perto do receptor alvo;
2. O aparelho, ao detectar a transmissão de um pacote, emite um ruído numa frequência próxima a de operação do protocolo, impedindo o receptor de interpretar corretamente o pacote;

3. O aparelho grava o sinal transmitido. O sinal gravado deve ser tratado com um filtro rejeita banda para remover o ruído criado pelo próprio aparelho. As etapas 2 e 3 são, efetivamente, um sequestro da transmissão;
4. A vítima (usuário), ao perceber que o sistema não executou seu comando, naturalmente pressiona o botão do transmissor novamente;
5. O aparelho detectar a segunda transmissão e repete os passos 2 e 3 para sequestrar esse novo pacote;
6. O aparelho envia o primeiro pacote sequestrado, executando a ação desejada pelo usuário (por exemplo, trancar o carro);
7. O aparelho, para qualquer futura transmissão, irá repetir os passos 2, 3 e 6. Dessa forma, o aparelho terá sempre gravado uma transmissão válida (o último pacote gerado pelo transmissor);
8. O atacante pressiona um botão no aparelho que permite que o último pacote guardado seja retransmitido, completando o ataque.

A única informação necessária para conduzir esse ataque é a faixa de frequência de operação do sistema, facilmente encontrada nos manuais e registros nas entidades reguladoras.

Como, no protocolo KeeLoq, o receptor espera que o valores de $COUNTER$ sejam sempre crescentes, é necessário que o aparelho sequestre o novo pacote e retransmita o antigo. Assim, um pacote gerado por um transmissor cadastrado é considerado válido até que um mais novo seja recebido e o receptor atualize o valor de $COUNTER_{RX}$. Não existe limite de tempo entre a data de geração do pacote atualmente armazenado no aparelho e a retransmissão do mesmo.

Ultimate KeeLoq®

Ultimate KeeLoq é uma tecnologia da Microchip apresentada em 2014 como alternativa mais segura ao Classic KeeLoq [16] e comercializado no chip de modelo MCS3142.

Assim como a versão original, o protocolo se baseia no conceito de “hopping code”. A principal diferença está no uso da cifra de bloco AES-128, reconhecida e aceita internacionalmente, ao invés da cifra KeeLoq [17, p. 4]. Com essa troca, a chave de encriptação passa de 64 para 128 bits e os ataques à cifra anterior não são mais efetivos.

O protocolo utiliza um contador de tempo incrementado a cada 0,25s para fazer com que os pacotes expirem, frustrando ataques que dependiam de sequestrar uma transmissão e enviá-la posteriormente.

Além disso, o protocolo usa código de autenticação (MAC) para proteger a transmissão contra modificações acidentais e maliciosas, garantindo sua integridade.

Descrição do Protocolo

Assim como a versão Classic, este protocolo é dividido em duas etapas: a de utilização e a de aprendizagem. A fase de utilização é executada cada vez que um botão é pressionado no transmissor e a fase de aprendizado quando se deseja cadastrar um transmissor no receptor.

Fase de Utilização

O transmissor armazena as seguintes informações:

- $SERIAL_{TX}$: identificador único do transmissor de 32 bits
- KEY_{TX}: chave de encriptação de 128 bits
- $COUNTER_{TX}$: contador de uso de 24 bits, incrementado a cada uso
- $TIME_{TX}$: contador de tempo de 32 bits, incrementado a cada 0,25s
- $POWERUP_{TX}$: contador de “power-ups” de 16 bits, incrementado cada vez que o transmissor perde o fornecimento de energia e é religado

A cada envio, um pacote de 192 bits (24 bytes) com o formato mostrado na Figura 8Figura 7 é gerado pelo transmissor.

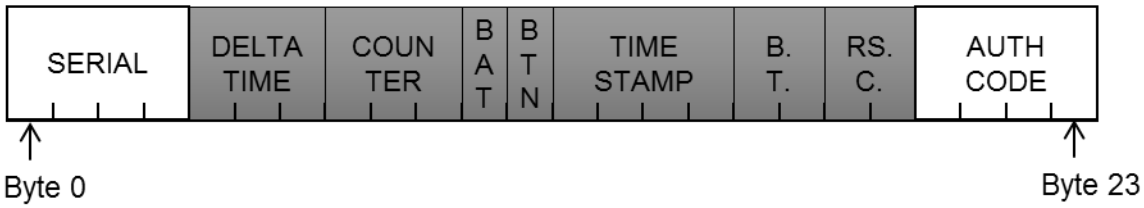

Figura 8 - Diagrama do pacote de utilização

Os 128 bits dos bytes de 4 a 19 são cifrados com o algoritmo AES-128 e chave KEY_{TX}.

Os campos presentes no pacote são:

- $SERIAL$: valor de $SERIAL_{TX}$
- $DELTA\ TIME$: tempo desde o último pressionamento de botão, útil para identificar gestos com duplo-clique
- $COUNTER$: valor atual de $COUNTER_{TX}$
- BAT: nível de bateria
- BTN: indicativo de qual botão foi pressionado
- $TIME\ STAMP$: valor atual de $TIMER_{TX}$
- $B.T.$, “button timer”: tempo de pressionamento do botão atual, útil para identificar gestos como clique longo
- $RS.C.$, “resync counter”: valor atual de $POWERUP_{TX}$
- $AUTH\ CODE$: código de autenticação (MAC) para identificar qualquer modificação (acidental ou intencional) e autenticar o transmissor, calculado como mostra a Figura 9, onde o bloco AES-128 é o algoritmo de cifra de bloco usando a chave KEY_{TX}.

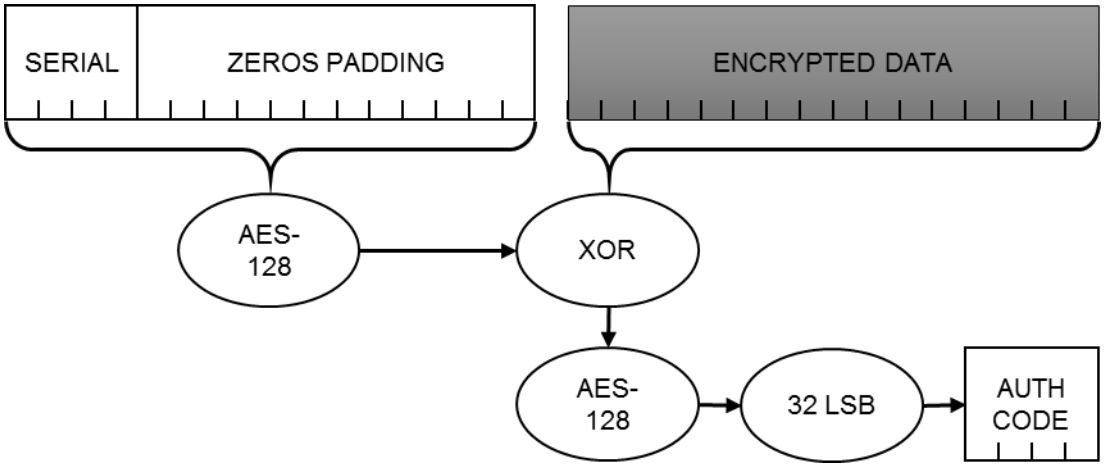

Figura 9 - Forma de cálculo do código de autenticação

O receptor, por sua vez, armazena as seguintes informações:

- $TIME_{RX}$: contador de tempo de 32 bits, incrementado a cada 0,25s
- TS_{RX}: conjunto de transmissores cadastrados. Cada elemento contém:
  - $SERIAL_{RX}$: identificador de 32 bits do transmissor
  - KEY_{RX}: chave de encriptação de 128 bits
  - $COUNTER_{RX}$: último valor recebido do contador de uso de 24 bits
  - $LAST\ TIME_{RX}$: valor de $TIME_{RX}$ quando o último pacote válido foi recebido
  - $TIME\ OFFSET_{RX}$: valor de diferença entre os contadores de tempo $TIME_{TX} - TIME_{RX}$
  - $POWERUP_{RX}$: último valor recebido do contador de “power-ups” de 16 bits

O receptor somente executa a ação se o pacote recebido passar pelas seguintes validações em ordem:

1. Existe algum transmissor cadastrado com o $SERIAL$ recebido;
2. O código de autenticação calculado a partir dos dados do pacote e KEY_{RX} confere com o recebido. Isso garante que o conteúdo não foi modificado em trânsito e que $KEY_{TX} = KEY_{RX}$;
3. Contador recebido é bem próximo do esperado: $COUNTER_{RX} < COUNTER \leq COUNTER_{RX} + 16$. Isso permite até 16 ativações acidentais entre duas ativações

válidas. Caso essa verificação falhe, mas $COUNTER_{RX} < COUNTER \leq COUNTER_{RX} + 32768$, o valor de $COUNTER$ é salvo temporariamente. Se o próximo pacote recebido trouxer $COUNTER + 1$ como valor do contador, o pacote é tido como válido. Essa resincronização permite o sistema se recuperar de grandes desvios no valor contador.

4. Pacote recebido não expirou. Esse passo envolve três situações diferentes:
    a. Se $RS.C. > POWERUP_{RX}$, o transmissor foi religado e, como ficou desligado por um tempo, o valor de $TIME\ STAMP$ não poderá ser conferido. Nesse caso, o pacote é tido como válido.
    b. Se $RS.C. < POWERUP_{RX}$, o pacote é inválido
    c. Se $RS.C. = POWERUP_{RX}$, o valor de $TIME\ STAMP$ deve estar próximo do esperado. Permite-se que o valor varie um pouco devido a imperfeições nos contadores de tempo de ambos os dispositivos. O tempo decorrido desde a última utilização é dado por $\Delta TIME_{RX} = TIME_{RX} - LAST\ TIME_{RX}$ e o máximo de discrepância aceita é $MAX\ DIFF = \alpha * \Delta TIME_{RX}$, onde α corresponde a porcentagem máxima de “drift” do hardware em questão. Dessa forma, o valor de $TIME\ STAMP$ deve ser próximo de $TIME_{RX} + TIME\ OFFSET_{RX} + \Delta TIME_{RX}$, com erro máximo de $MAX\ DIFF$.

Se o pacote for aceito, o valor de $COUNTER_{RX}$, $LAST\ TIME_{RX}$, $TIME\ OFFSET_{RX}$ e $POWERUP_{RX}$ são atualizados de acordo, efetivamente tornando inválido qualquer outro pacote com um valor de contador de utilização menor.

O receptor também tem um contador de tempo, que deve incrementar na mesma frequência do que nos transmissores. A Microchip sugere usar uma bateria de backup para manter o contador rodando em caso de falta de energia no receptor. Caso a bateria de backup acabe ou não esteja instalada, o contador do receptor irá se dessincronizar e todos os receptores terão de ser cadastrados novamente.

Fase de Aprendizagem

A fase de aprendizagem é necessária para que o receptor tome conhecimento de um transmissor. Ao habilitar o modo de aprendizado em ambos, o transmissor envia um pacote como apresentado na Figura 10.

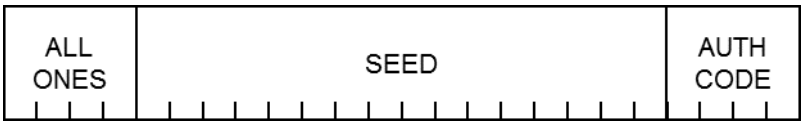

Figura 10 - Diagrama do pacote de aprendizado

Os campos presentes no pacote são:

- $ALL\ ONES$: 32 bits com valor 1
- $SEED$: valor de 128 bits gerado aleatoriamente, pré-programado em cada transmissor
- $AUTH\ CODE$: código de autenticação gerado com KEY_{TX}, da mesma forma que no pacote de utilização

O receptor irá então derivar a chave com base no valor de “seed” e a chave de fabricante que este possui pré-gravado. A documentação consultada, no entanto, não descreve exatamente como esse processo é executado.

Ataques

Não foi possível encontrar publicações de ataques a esse protocolo, muito provavelmente por ter sido lançado há menos de 2 anos.

Capítulo 4 – Resultados e Discussões

Análise Código Fixo

A solução de código fixo pode ser implementada num hardware muito simples e barato. Porém, ela não apresenta segurança nenhuma, nem mesmo resistência a busca exaustiva (força bruta), que pode ser executada com sucesso em menos de 10 segundos.

É difícil imaginar uma aplicação onde um nível tão baixo de segurança seja suficiente. Isso levanta inclusive questionamentos sobre a ética das empresas que ainda comercializam esse tipo de produto. Os clientes que eventualmente acabam instalando esses sistemas pouco provavelmente estão cientes do grau tão baixo de segurança efetivamente fornecido.

O sistema é dito tão quebrado por duas principais razões:

1. O segredo tem muito poucos bits, geralmente de 10 a 12, quando o recomendado é de no mínimo 112 bits [18]
2. O sistema não é seguro contra ataques de repetição. Basta gravar uma transmissão e repeti-la posteriormente para burlar o sistema.

Análise Classic KeeLoq®

O protocolo KeeLoq apresenta graves falhas de segurança conhecidas, em diversos componentes do design e faz uso de práticas antiquadas e não recomendadas há vários anos pela academia e agências reguladoras. Alguns exemplos de práticas duvidosas:

- A cifra em bloco KeeLoq usa chaves de 64 bits, ou seja, há 2^{64} diferentes chaves. Esse é um valor muito inferior ao recomendado de pelo menos 2^{112} [18]
- O protocolo não garante a integridade da transmissão, ou seja, não há nenhum mecanismo de defesa contra modificações maliciosas dos pacotes. O recomendado é o uso de alguma primitiva MAC (“Message Authentication Code”) ou AE (“Authenticated Encryption”)

- O protocolo usa a chave de fabricante em todos os dispositivos do sistema, abrindo caminhos para ataques que afetem todos os dispositivos de um fabricante, uma vez que se tenha acesso à chave, como demostrado num ataque prático acima
- O protocolo não faz uso de nenhum mecanismo de expiração do pacote, permitindo que o mesmo seja sequestrado e utilizado maliciosamente no futuro
- O protocolo utiliza construções proprietárias, no lugar de algoritmos bem conhecidas e sustentadas por várias pesquisas e entidades respeitadas (como AES)

Análise Ultimate KeeLoq®

As construções primitivas empregadas no design do protocolo (a saber, AES-128 e CBC-MAC) são bem sólidas, amplamente conhecidas, estudas e aprovadas por especialistas.

A nova versão corrige vários problemas relatados na anterior, como:

- Tamanho da chave, de 64 para 128 bits
- Uso de MAC para garantir a integridade da transmissão
- Pacotes expiram depois de alguns segundos
- Emprego de bons algoritmos, padrões na indústria
- Seed de 128 bits para geração da chave

O protocolo foi anunciado a pouco mais de um ano e não parece não ter recebido muita atenção dos críticos ainda. Porém, pode-se identificar alguns pontos do design do protocolo que não seguem as melhores práticas, que podem resultar em falhas no modelo de segurança. Esses pontos não implicam que há de fato um problema, mas são prováveis pontos de problema:

- Uso da mesma chave para cifrar e autenticar o pacote. O recomendado é que se usem chaves distintas e não relacionadas, cada uma para um propósito. Para isso, deve-se usar algoritmos de derivação de chaves (KDF) [19].

- Uso de chave do fabricante: uma chave global gravada em todos os receptores. Um atacante em poder dessa chave, inclusive o próprio fabricante, pode ter mais ângulos de ataques. Um efeito semelhante foi descrito em “Ataque prático na implementação em software” para o protocolo Classic.

Capítulo 5 – Sugestão de Novo Protocolo

Este capítulo irá definir um protocolo adequado aos parâmetros de segurança desejados. Para tal, primeiro será apresentado um protocolo de autenticação introduzido em 2010 e posteriormente adequado ao caso de uso em questão.

Protocolo SCRAM

"*Salted Challenge Response Authentication Mechanism*" (SCRAM) é um protocolo de autenticação moderno baseado em usuário e senha, para autenticar um cliente em um servidor. Ele é utilizado por serviços de e-mail (SMTP, IMAP), chat (XMPP) e bancos de dados (MongoDB), entre outros [20] [21]. A norma RFC 5802 define o funcionamento e uso do protocolo [22].

O SCRAM oferece algumas características interessantes, como:

- A senha do usuário é armazenada de forma segura, na forma de múltiplas iterações da *hash* SHA-1 e um sal único;
- O cliente demonstra que tem posse da senha sem que ela seja revelada durante a comunicação;
- Um servidor mal-intencionado não consegue extrair nenhuma informação do cliente;
- Um cliente mal-intencionado não consegue extrair nenhuma informação do servidor;
- Autenticação mútua: ao final do processo, o cliente consegue verificar se o servidor não é um impostor e realmente tem a autoridade de verificar a autenticação;
- As informações armazenadas no servidor não são suficientes para completar a autenticação como cliente, ou seja, um agente externo com posse desses dados não consegue burlar o sistema.

O protocolo é composto de quatro mensagens, descritas abaixo e mostradas na Figura 11:

1. Início Cliente: o cliente envia o nome do usuário e um valor arbitrário que deve ser diferente a cada execução (“nonce cliente”);
2. Início Servidor: o servidor gera um valor arbitrário único (“nonce servidor”) e define $nonce = nonce\ cliente \mid nonce\ servidor$. Como resposta, ele envia $nonce$, e os parâmetros de processamento da senha (sal e $iterações$);
3. Fim Cliente: com os parâmetros recebido, o cliente processa a senha $senha' = PBKDF(senha, sal, iterações)$, gera uma chave $key_c = HMAC(senha', \text{"Client Key"})$ e a utiliza para gerar uma prova de que conhece a senha: $prova\ cliente = key_c \oplus HMAC(H(key_c), auth)$. Onde $auth = username|nonce|sal|iterações$, PBKDF, HMAC e H são funções de hash;
4. Fim Servidor: o servidor verifica a prova do cliente e gera uma prova para garantir o cliente de que a autenticação foi executada com sucesso $prova\ servidor = HMAC(key_s, auth)$, onde $key_s = HMAC(senha', \text{"Server Key"})$.

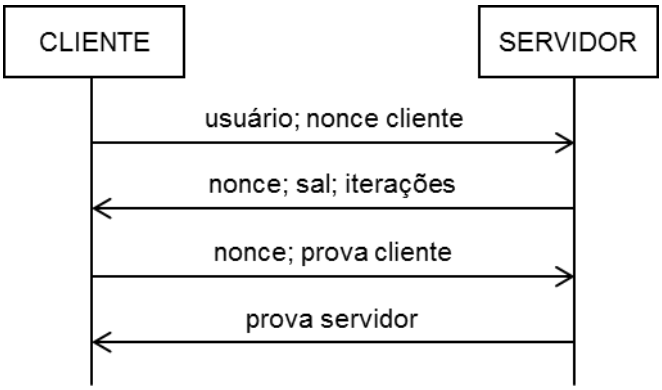

Figura 11 - Sequência de mensagens do protocolo SCRAM

Pode-se notar que as provas geradas pelo cliente e pelo servidor dependem dos valores de $nonce$ escolhidos por cada parte. Dessa forma, o protocolo é livre de ataques de repetição, já que a cada nova interação, esses valores são trocados.

O servidor só precisa armazenar os seguintes valores: $H(key_c)$, key_s, sal e $iterações$. Dessa forma, a senha não é armazenada diretamente e, mesmo que um atacante consiga acessar essas informações, elas não são suficientes para obter a senha nem completar uma autenticação com sucesso, já que para gerar $prova\ cliente$ é necessário ter key_c. Como $prova\ cliente = key_c \oplus HMAC(H(key_c), auth)$, o servidor consegue validar a prova do cliente verificando se $H(key_c) = H(prova\ cliente \oplus HMAC(H(key_c), auth))$.

No entanto, um servidor mal-intencionado pode manter conhecimento de key_c após uma autenticação bem-sucedida, já que $key_c = prova\ cliente \oplus HMAC(H(key_c), auth)$. Assim, SCRAM não é um protocolo *zero-knowledge*, pois o cliente, ao provar que tem posse de key_c, também o releva [23].

Adequando SCRAM

Nesta seção, o protocolo SCRAM descrito anteriormente será adequado ao caso de uso em questão. Note que, no caso de autenticação por radiofrequência, a central atua como servidor e o periférico como cliente. As duas principais modificações sugeridas são:

1. Remover $prova\ servidor$. O protocolo SCRAM original insere essa funcionalidade pois a comunicação continua entre as partes após a autenticação. Assim, o cliente poderia garantir a autenticidade do servidor. No caso de interesse deste estudo, a comunicação se encerra logo após a autenticação, portanto o cliente não faria nada com essa informação.
2. Trocar $senha$ por uma chave criptográfica. O protocolo SCRAM opera sobre um par $(usuário, senha)$ e assume que a senha foi escolhida por um humano e, por isso, não possui aleatoriedade suficiente. Como consequência, o protocolo faz uso de primitivas como PBKDF ("*Password-based Key Derivation Function*"). No caso em estudo, o segredo compartilhado é uma chave com aleatoriedade (entropia) suficiente para dispensar este tratamento.

Ao aplicar essas duas modificações ao protocolo, simplificando as construções necessárias, chega-se a seguinte proposta de sequência de mensagens, ilustrado na Figura 12:

1. Início Cliente: o cliente envia o serial do dispositivo e um valor aleatório que deve ser diferente a cada execução ("nonce cliente");
2. Início Servidor: o servidor gera um valor aleatório ("nonce servidor") e responde com $nonce = nonce\ cliente \mid nonce\ servidor$;
3. Fim Cliente: com os parâmetros recebido, o cliente gera uma prova de que conhece key_c: $prova\ cliente = key_c \oplus HMAC(H(key_c), auth)$. Onde $auth = serial|nonce|$, HMAC e H são funções de hash.

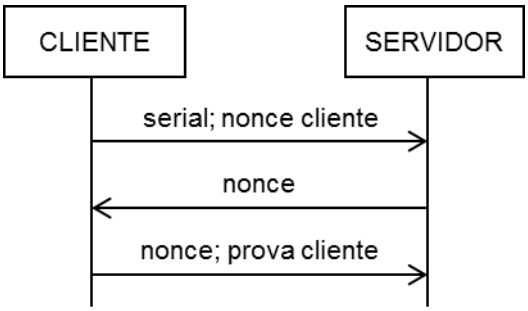

Figura 12 - Sequência de mensagens do protocolo baseado em SCRAM

Note que a remoção da prova do servidor tira a necessidade da existência da chave key_s e, com isso, não é mais necessário derivar a chave key_c. Assim, a autenticação se resume em o cliente convencer o servidor de que tem posse de key_c. O servidor armazena $H(key_c)$ e verifica a prova da mesma forma exibida no protocolo original: $H(key_c) = H(prova\ cliente \oplus HMAC(H(key_c), auth))$.

Note também que a troca de uma senha por uma chave tira a necessidade de mecanismos defensivos contra a possível falta de entropia da senha de um usuário. Dessa forma, pode-se remover sal, $iterações$, $senha$ e $senha'$. E, por ajuste de nomenclatura, $username$ dá lugar a $serial$.

Usando nonces de 40 bits, um serial de 32 bits e adicionando espaço para 8 bits de dados arbitrários (D) na primeira mensagem do cliente, os pacotes transmitidos podem ser organizados como apresentado na Figura 13. O pacote início do cliente é preenchido com zeros para que tenha o mesmo tamanho do pacote de fim do cliente.

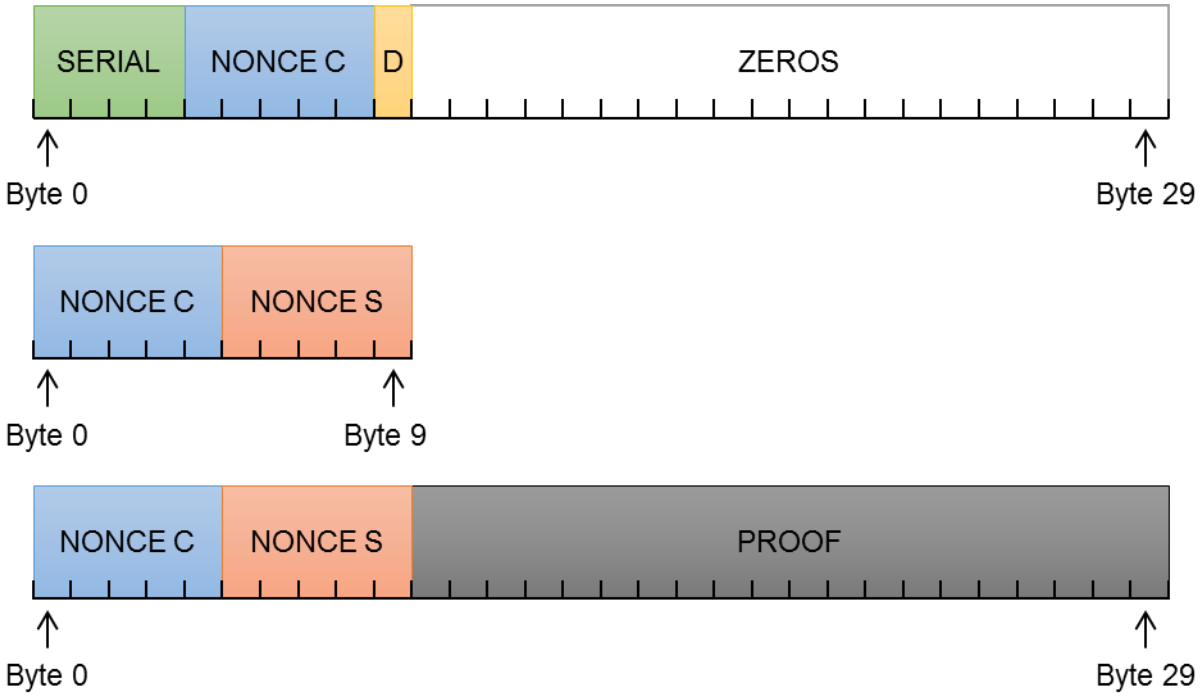

Figura 13 - Pacotes início do cliente, início do servidor e fim do cliente

Capítulo 6 – Programação VHDL

Esse capítulo irá tratar da parte prática relativa a este estudo. Nesta parte, as soluções Código Fixo, Classic KeeLoq e SCRAM modificado serão implementadas em hardware numa FPGA usando VHDL. As três implementações serão comparadas com relação à quantidade de elementos lógicos utilizados, energia consumida a cada envio entre outras métricas.

Este estudo considera apenas a implementação da fase de utilização dos protocolos, com o foco no periférico/cliente, já que tem maiores restrições de energia, tamanho e preço.

Fundamentos

O principal componente de hardware utilizado na implementação foi a placa de desenvolvimento DE1-SoC da marca Terasic, que possui a FGPA Cyclone V da Altera. Para programação foi utilizado o software Quartus Prime Lite 16.0 e o simulador Model-Sim Altera 10.4d.

Uma visão de alto nível das entidades implementadas e suas conexões é dada pela Figura 14. As entidades transmissor e receptor são as mesmas para todas as soluções, enquanto que as entidades periférico e central são responsáveis pela lógica característica de cada protocolo.

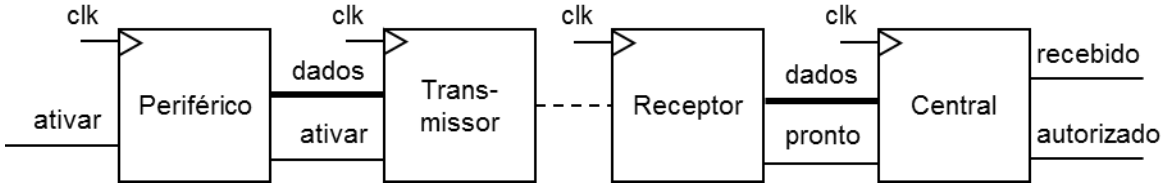

Figura 14 - Diagrama das entidades implementadas

O fluxo de execução segue da esquerda para a direita, começando quando o botão “ativar” do periférico é pressionado. A entidade “periférico” monta o pacote a ser transmitido e o disponibiliza por completo no barramento de dados. A entidade “transmissor” envia o pacote usando uma codificação PWM, explicada mais adiante. A entidade “receptor”

decodifica a transmissão digital e disponibiliza o pacote no barramento de dados à direita. Por fim, a entidade “central” aplica a lógica do protocolo para decidir se o acesso é autorizado. A saída “recebido” fica ativa por 1s segundo quando um pacote é recebido e a saída “autorizado” quando o pacote é tido como válido.

A transmissão é serial e composta de quatro regiões: preâmbulo, cabeçalho, corpo e guarda. A Figura 15 demostra uma transmissão de 8 bits. O preâmbulo (em vermelho) é composto de 16 ciclos em alta intercalados com 16 em baixa. O cabeçalho é composto de 8 ciclos em baixa. O corpo transmite cada bit sequencialmente, com três ciclos por bit. Para um 0 usa-se a sequência alta, alta, baixa (em azul); para um 1 usa-se a sequência alta, baixa, baixa (em verde). Por fim, um período de guarda de 16 ciclos.

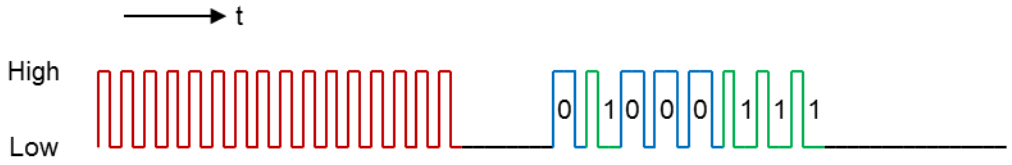

Figura 15 - Exemplo da transmissão dos bits 01000111

O receptor foi implementado como uma máquina de estados, representada na Figura 16. Os estados com contorno vermelho têm uma transição possível não explicitada. Esses são casos de erro e sempre levam para o estado inicial “OFF”. Essas transições não estão representadas para dar mais clareza ao desenho.

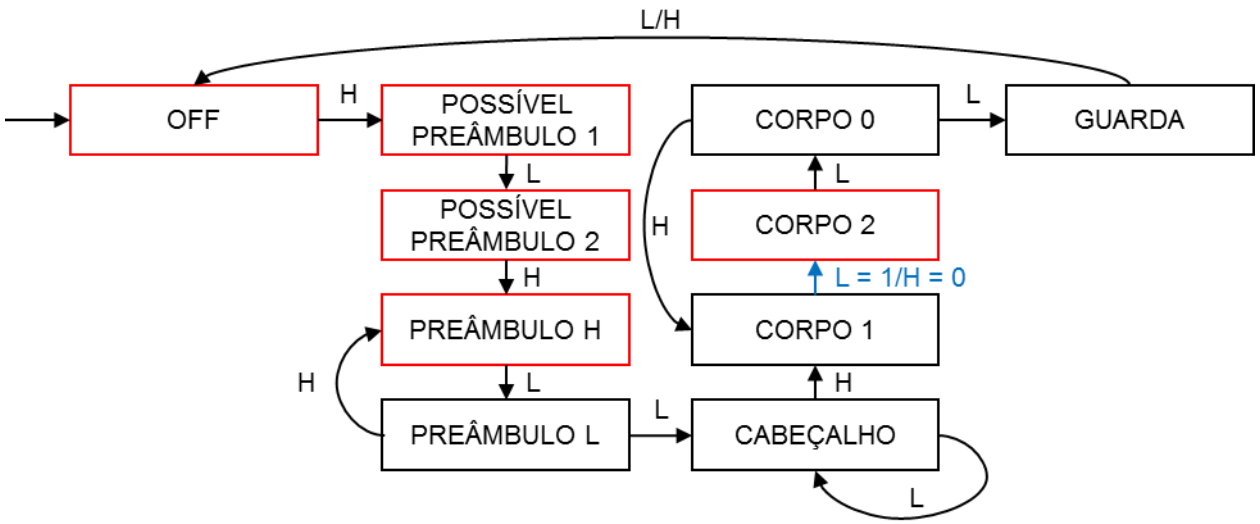

Figura 16 - Máquina de estados do receptor de codificação PWM

A máquina de estados foi construída de forma que, a partir de qualquer estado, após no máximo 5 sequências de [alta, baixa] ela entre no ciclo entre “PREÂMBULO H” e “PREÂMBULO L”. Note que a transmissão marcada em azul representa o recebimento de um novo bit.

O código dos componentes de transmissão está acessível no endereço https://github.com/sitegui/TG/tree/master/PWM.

Código Fixo

A implementação do código fixo é a mais direta. O periférico é composto de um componente transmissor (PWM_TX) com entrada fixa. A central é composta de um componente receptor (PWM_RX) com uma simples verificação de igualdade entre o código recebido e o esperado.

Código fonte: https://github.com/sitegui/TG/tree/master/fixedCode

Classic KeeLoq®

A implementação do protocolo comercial KeeLoq envolve não somente a transmissão e recepção de dados, mas também a geração e verificação dos mesmos.

O periférico deve cifrar 32 bits do conteúdo do pacote com o algoritmo KeeLoq e calcular 2 bits de verificação com o algoritmo CRC, como explicado na subseção Fase de Utilização. Para essas tarefas foram criados componentes isolados e reutilizáveis, chamados de entidades na nomenclatura VHDL. A hierarquia desses componentes está representada na Figura 17. O número entre parênteses representa a quantidade de unidades lógicas ALM (Adaptive Logic Module) utilizadas na síntese do componente e de seus descendentes.

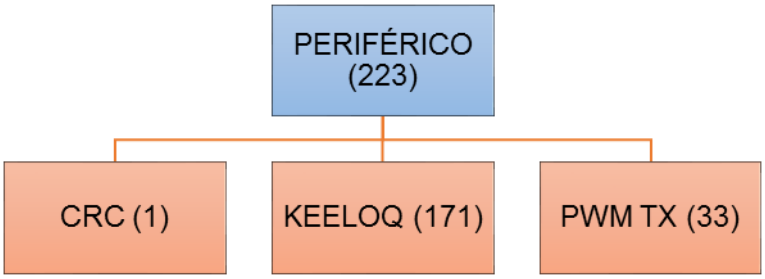

Figura 17 - Hierarquia dos componentes na implementação KeeLoq

A lógica principal do periférico e da central foi modelada numa máquina de estados, representada na Figura 18 e Figura 19 respectivamente. O número entre parênteses representa o número de ciclos de clock para a execução de cada etapa. As etapas que dependem de estímulo externo, como o acionamento de um botão pelo usuário, têm um “?” no lugar do número. As transições em vermelho representam erros na execução do protocolo, voltando ao estado de “repouso”.

Para o periférico foram considerados os seguintes estados:

- Off: periférico em modo de espera, aguardando o pressionamento de algum botão
- Encrypt: aplicação dos 528 rounds do algoritmo KeeLoq

- Crc: aplicação dos 65 rounds do algoritmo CRC
- Tx: transmissão dos 69 bits do pacote gerado

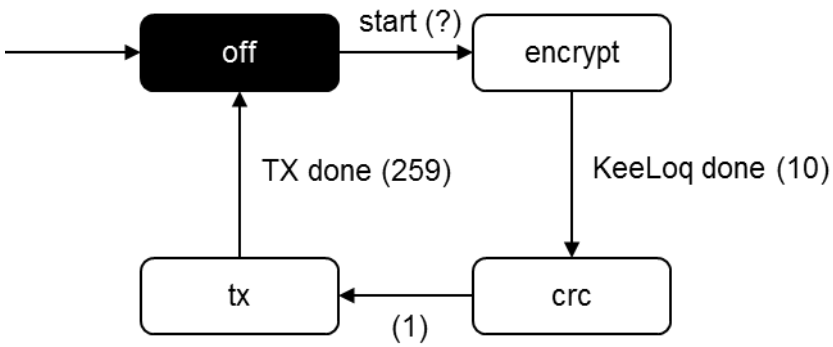

Figura 18 - Máquina de estados da implementação do periférico KeeLoq

Para a central foram considerados os seguintes estados:

- Rx: a central em modo de espera, aguardando o recebimento de algum pacote
- Crc: verificação do CRC
- Decrypt: aplicação reversa da cifra KeeLoq e comparação do valor de contador de uso

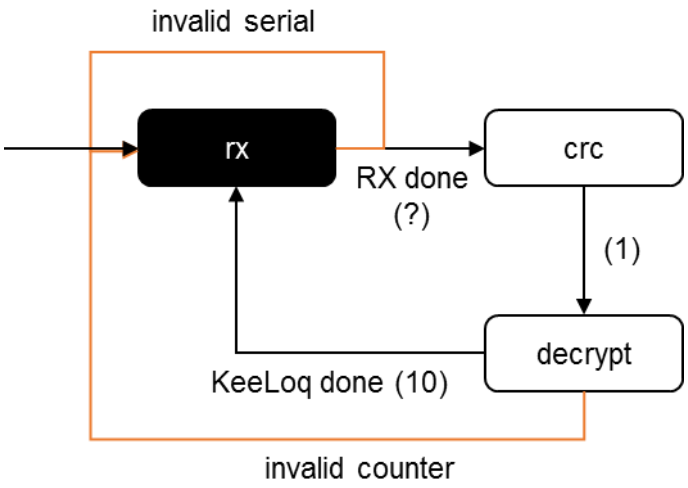

Figura 19 - Máquina de estados da implementação da central KeeLoq

Código fonte: https://github.com/sitegui/TG/tree/master/keeLoq

SCRAM Modificado

A implementação do protocolo sugerido no cliente faz uso de 5 subcomponentes, como representado na Figura 20. O número entre parênteses representa a quantidade de unidades lógicas (ALM) utilizadas.

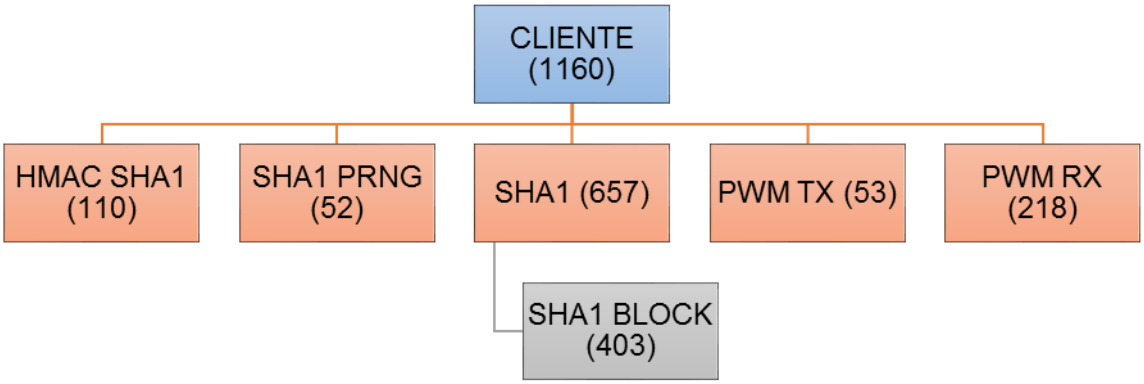

Figura 20 - Hierarquia dos componentes na implementação SCRAM modificado

O componente SHA1 implementa a função de hash de mesmo nome. Para simplificar a implementação, este componente pode operar sobre somente dois tamanhos de entrada: 160 ou 672 bits, ao contrário do algoritmo original de SHA1, que é definido para tamanhos de até 2^{64} bits. Este componente faz uso de um subcomponente para processar um bloco, como definido na especificação oficial.

Os componentes de transmissão (PWM TX) e recepção (PWM RX) são configurados para operar com 240 e 80 bits, respectivamente.

O componente HMAC-SHA1 implementa o algoritmo de mesmo nome para uma entrada de 160 bits.

O componente SHA1 PRNG é um gerador de números pseudoaleatórios baseado no algoritmo SHA1, responsável por gerar os nonces de forma imprevisível. O algoritmo SHA1

foi escolhido para reaproveitar o mesmo circuito já disponível para o componente HMAC-SHA1, compartilhando recursos. O funcionamento básico deste componente está ilustrado na Figura 21. A cada vez que se deseja 40 bits pseudoaleatórios, o algoritmo SHA1 é aplicado ao estado interno de 160 bits e 40 bits do resultado são retornados. O estado interno deve ser inicializado com 160 bits aleatórios colhidos de alguma fonte de grande entropia.

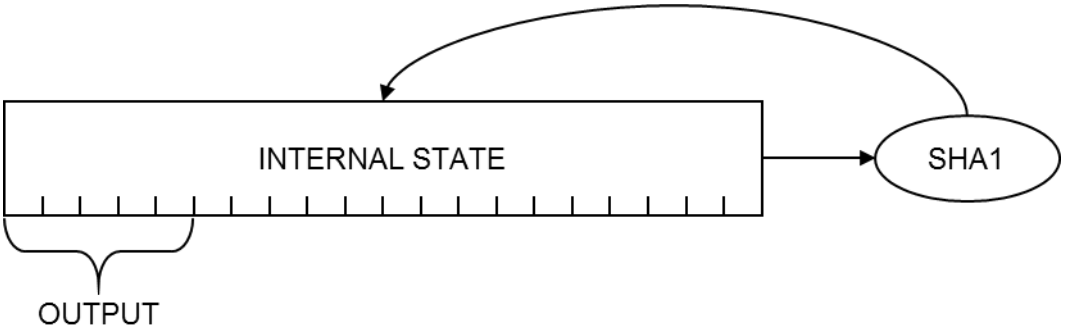

Figura 21 - Funcionamento do SHA1 PRNG

As máquinas de estados do cliente e do servidor estão representadas na Figura 22 e Figura 23. O número entre parênteses representa o número de ciclos de clock para a execução de cada etapa. As etapas que dependem de estímulo externo, como o acionamento de um botão pelo usuário, têm um “?” no lugar do número. As transições em vermelho representam erros na execução do protocolo, voltando ao estado de “repouso”.

Para o cliente foram considerados os seguintes estados:

- Startup1 e startup2: inicialização do dispositivo
- Off: repouso, à espera de algum botão ser pressionado
- Waitnonce: geração do nonce do cliente
- Tx1: transmissão do pacote de início do cliente
- Rx: aguardando pacote de início do servidor
- Proof: cálculo da prova
- Tx2: transmissão do pacote de fim do cliente

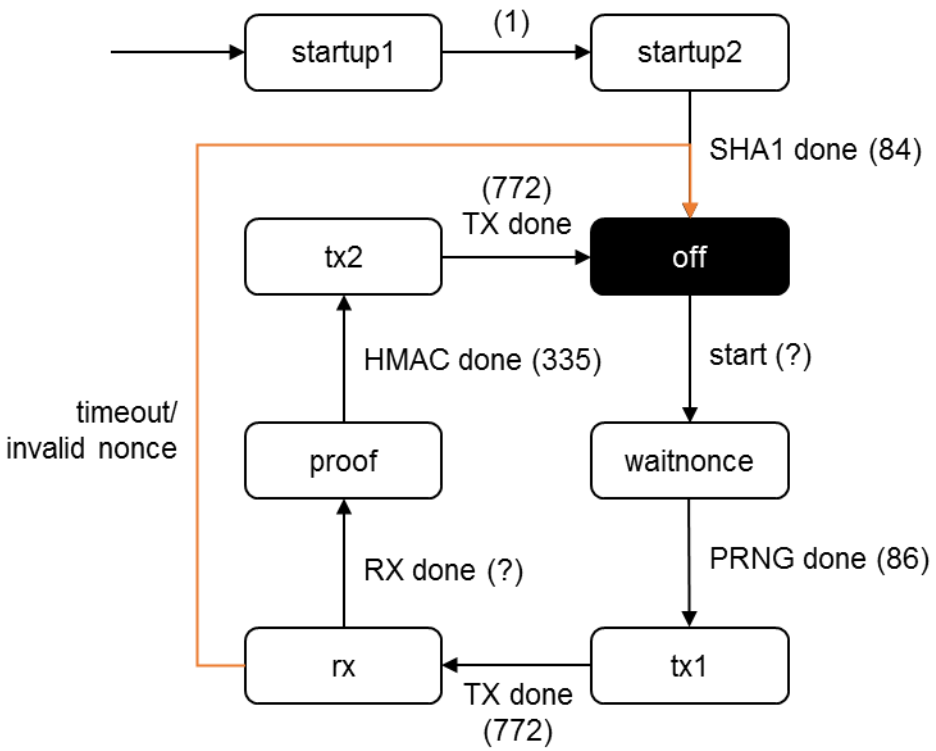

Figura 22 - Máquina de estados da implementação do cliente SCRAM

Para o servidor foram considerados os seguintes estados:

- Rx1: repouso, à espera da primeira transmissão
- Waitnonce: geração do nonce do servidor
- Tx: transmissão do pacote de início do servidor
- Rx2: aguardando pacote de fim do cliente
- Check1 e check2: verificação da prova do cliente

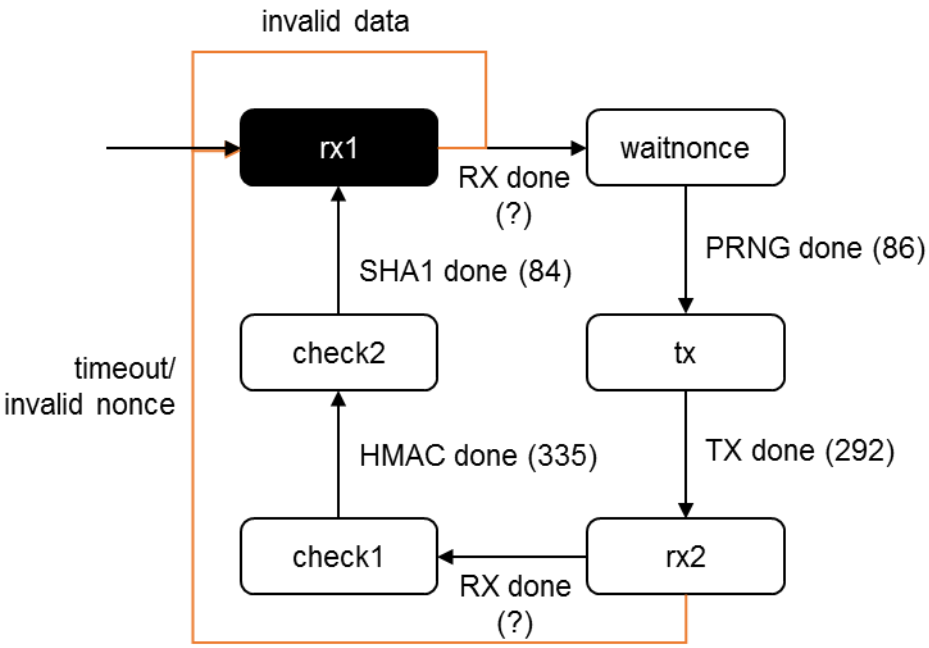

Figura 23 - Máquina de estados da implementação do servidor SCRAM

Código fonte: https://github.com/sitegui/TG/tree/master/scram

Capítulo 7 – Conclusões

Dos três sistemas comerciais de autenticação apresentados nesse trabalho, os dois primeiros podem ser considerados como inseguros. O terceiro corrige vários problemas identificados na versão anterior e tem um nível de segurança aparentemente adequado, mas ainda faltam estudos de terceiros para corroborar essa afirmação.

Percebe-se que a indústria começou a responder recentemente aos problemas já há muito tempo apontados por pesquisadores. No entanto, é esperado que esses novos produtos demorem a ser amplamente utilizados, até mesmo porque as versões antigas ainda são comercializadas e o consumidor pouco sabe dos problemas associados a esses produtos.

A segunda parte deste trabalho, sugeriu e explorou um protocolo alternativo, baseando-se em protocolos de autenticação já em uso em outras aplicações e com sólidos resultados de segurança. Isso permite ter uma melhor confiança no resultado, mas somente mais anos de estudos e aplicação podem cristalizar (ou desbancar) essa confiança.

Os quatro protocolos citados são comparados lado a lado na Tabela 3.

Tabela 3 - Comparação dos protocolos

Parâmetro	**Código fixo**	**KeeLoq**	**Ultimate KeeLoq**	**SCRAM modificado**
Pacotes enviados por uso	1	1	1	3
Periférico precisa receber dados	Não	Não	Não	Sim
Bits enviados a cada uso	10	69	192	560

Parâmetro	Código fixo	KeeLoq	Ultimate KeeLoq	SCRAM modificado
Tamanho da chave (em bits)	10	64	128	160
Primitiva criptográfica	Nenhum	KeeLoq	AES-128	SHA-1
Segurança	Nenhuma	Fraca	Desconhecida, mas com boa primitiva	
ALM usadas na implementação do periférico/cliente	16	223	(Não medido)	1160
Ciclos de clock por uso	74	273	(Não medido)	2735
Tipo de proteção contra replay attack	Nenhuma proteção	Contador de uso	Relógio	Nonce
Suscetível a replay attack	Sim	Sim, com jam-and-replay	Não, somente por alguns segundos	Não, o nonce do servidor muda a cada uso
Faz uso de chave de fabricante	Não	Sim	Sim	Não
Observações	-	Contador de uso pode	Periférico e central precisam contar o tempo. Central	-

Parâmetro	Código fixo	KeeLoq	Ultimate KeeLoq	SCRAM modificado
		sair de sincronia	precisa continuar contagem mesmo quando sem energia	

Como resultado final deste trabalho, o protocolo sugerido tem boas perspectivas de ser aplicável na prática e de melhorar a situação da segurança dos sistemas de autenticação por rádio frequência.

Bibliografia

[1] S. Kamkar, “DRIVE IT LIKE YOU HACKED IT,” em *DEFCON 23*, 2015.

[2] Wikipedia, "De Bruijn sequence," [Online]. Available: https://en.wikipedia.org/wiki/De_Bruijn_sequence. [Accessed junho 2016].

[3] S. Kamkar, “OpenSesame,” junho 2015. [Online]. Available: http://samy.pl/opensesame/. [Acesso em junho 2016].

[4] Microchip Technology Inc, “US patent 5517187, Bruwer, Frederick J.; Smit, Willem & Kuhn, Gideon J., "Microchips and remote control devices comprising same", issued 1996-05-14, assigned to Microchip Technology Inc,” 05 1996. [Online]. Available: http://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US5517187.

[5] Microchip, “Embedded Security | Microchip Technology Inc.,” [Online]. Available: http://www.microchip.com/design-centers/embedded-security.

[6] Microchip, “HCS410 KEELOQ® Code Hopping Encoder and Transponder,” 2011. [Online]. Available: http://ww1.microchip.com/downloads/en/DeviceDoc/40158F.pdf.

[7] Microchip, “HCS500 KEELOQ® Code Hopping Decoder,” 2011. [Online]. Available: http://ww1.microchip.com/downloads/en/DeviceDoc/40153D.pdf.

[8] T. Kasper, "SECURITY ANALYSIS OF PERVASIVE WIRELESS DEVICES," Bochum, 2011.

[9] S. Indesteege, N. Keller, O. Dunkelman, E. Biham and B. Preneel, "A Practical Attack on KeeLoq," 2010.

[10] M. E. Hellman, "A Cryptanalytic Time-Memory Trade-Off," *IEEE Transactions on Information Theory vol. 26,* pp. 401-406, 1980.

[11] N. T. Courtois and G. V. Bard, "Algebraic and Slide Attacks on KeeLoq," Cryptology ePrint Archive, 2007.

[12] A. Bogdanov, "Cryptanalysis of the KeeLoq block cipher," Cryptology ePrint Archive, 2007.

[13] A. Bogdanov, “Attacks on the KeeLoq Block Cipher and Authentication Systems,” em *3rd Conference on RFID Security 2007 (RFIDSec 2007)*, 2007.

[14] N. T. Courtois, G. V. Bard e D. Wagner, “Algebraic and Slide Attacks on KeeLoq,” em *Proceedings of Fast Software Encryption 2008*, Verlag, 2008.

[15] S. Whyte, "Jam Intercept and Replay Attack against Rolling Code Key Fob Entry Systems using RTL-SDR," 2014. [Online]. Available: http://spencerwhyte.blogspot.com.br/2014/03/delay-attack-jam-intercept-and-replay.html.

[16] Microchip Technology Inc., "Introduction to Ultimate KEELOQ® Technology," 2014.

[17] E. Barker and A. Roginsky, "Transitions: Recommendation for Transitioning the Use of Cryptographic Algorithms and Key Lengths," 2015.

[18] E. Barker, "Recommendation for Key Management - NIST SP 800-57 part 1 rev. 4," 2016.

[19] L. Chen, "Recommendation for Key Derivation Using Pseudorandom Functions," 2009.

[20] Wikipedia, “Salted Challenge Response Authentication Mechanism,” [Online]. Available: https://en.wikipedia.org/wiki/Salted_Challenge_Response_Authentication_Mechanism.

[21] MongoDB, “SCRAM-SHA-1 -- MongoDB Manual 3.2,” [Online]. Available: https://docs.mongodb.com/manual/core/security-scram-sha-1/.

[22] C. Newman, A. Menon-Sen, A. Melnikov e W. N., “RFC5802 - Salted Challenge Response Authentication Mechanism (SCRAM),” Julho 2010. [Online]. Available: https://tools.ietf.org/html/rfc5802.

[23] Wikepedia, “Zero-knowledge proof,” [Online]. Available: https://en.wikipedia.org/wiki/Zero-knowledge_proof.